

Project Report: Multi-User Remote OS-Shell (O-Shell)

Name: Jaswanth Gangavarapu

Roll No: BT2024010

1. Executive Summary

O-Shell is a robust Unix-like command-line interpreter developed in C to demonstrate core Operating System principles. The system integrates process management, concurrency control via mutexes, role-based access control (RBAC), and network-based Inter-Process Communication (IPC). A key feature is the "Remote Shell" (RSH) capability, allowing authenticated users to execute commands on a remote server securely.

2. Directory Structure

The project is organized into a modular structure to ensure maintainability:

- **data/:** Contains history.log for command persistence and users.txt for credential storage.
- **include/:** Public header files including shell.h, server.h, and client.h.
- **src/:** Implementation files:
 - main.c: Entry point and shell loop initialization.
 - shell.c: Core logic for parsing, job control, and signal handling.
 - server.c: TCP server for remote connection handling.
 - client.c: RSH client for remote command execution.
- **Makefile:** Script for automated project compilation.

3. Mandatory OS Concept Implementation

3.1 Role-Based Authorization (Section 4.1)

The system implements a login mechanism that validates users against data/users.txt. Permissions are enforced through three distinct roles:

- **Admin:** Full system access.
- **User:** Restricted from process-level commands such as kill.
- **Guest:** Restricted to basic read-only commands (e.g., ls, pwd, echo).

3.2 File Locking & Data Consistency (Sections 4.2 & 4.4)

To prevent race conditions when multiple shell instances write to history simultaneously, the project uses **Advisory Locking** via fcntl().

- A **Write Lock** (F_WRLCK) is acquired during append_history().
- A **Read Lock** (F_RDLCK) is acquired during show_history().
- This ensures data integrity and prevents "dirty reads" or "lost updates".

3.3 Concurrency Control (Section 4.3)

Background job management is protected by a global mutex (pthread_mutex_t jobs_lock). This ensures that asynchronous signal handlers (like SIGCHLD) and the main execution thread do not conflict while modifying the shared job array.

3.4 Socket Programming & IPC (Sections 4.5 & 4.6)

- **Sockets:** The shell supports a --server mode using the TCP/IP stack (AF_INET) to accept remote client connections.
- **Pipes:** Implemented via pipe() and dup2() to support multi-stage command chaining.
- **Signals:** Custom handlers for SIGINT (Ctrl+C) and SIGTSTP (Ctrl+Z) manage process groups and terminal control.

4. Challenges & Technical Solutions

The Current Working Directory (CWD) Conflict

- **Challenge:** Changing directories via the `cd` command caused the shell to lose access to its `data/` folder.
- **Solution:** Implemented a **Global Base-Path Anchor**. At startup, `getcwd()` captures the absolute path of the project root into a `BASE_DIR` variable. All internal file lookups now use absolute paths derived from this anchor.

Signal Handler Re-entrancy & EINTR

- **Challenge:** Signals were causing the `fgets()` function to return prematurely with an error, making the shell appear "stuck".
- **Solution:** Integrated **errno saving and restoring** in signal handlers and added an **EINTR check** in the main input loop to resume the prompt seamlessly after an interruption.

5. Build & Execution Instructions

1. **Build:** Execute the `make` command in the root directory.
2. **Local Shell:** Run `./oshell` and enter valid credentials.
3. **Server Mode:** Run `./oshell --server` to listen for remote connections on the configured port.
4. **Remote Execution:** Use the command `rsh <host> <port> <command>` from within a local shell session.

6. Conclusion

O-Shell successfully fulfills the requirements of the Operating Systems lab guidelines by providing hands-on implementation of concurrency management, secure authorization, and network communication. The modular design ensures it is a scalable platform for future OS experimentation.